JAX

Complete Personal AI Operating Layer - Architecture, Security Model, Database Schema, APIs, Device Nodes, Memory, Agents, and Starter Code

Mission: turn JAX into Jason's private cross-device AI that can understand approved activity on a personal laptop, work computer, and Android phone; maintain persistent memory; coordinate specialized agents; produce handoffs; and safely execute authorized actions.

Design principle	Decision
User-centric	JAX serves Jason first. Business systems become optional skills/workspaces.
Local-first context	Collect structured events locally; do not continuously upload raw screens or keystrokes.
Persistent memory	Separate facts, projects, episodes, tasks, documents, and event history.
Permissioned action	Observation and execution have explicit capability levels.
Cross-device continuity	Every device writes to the same JAX Core and can resume another device's work.
Model-agnostic	Cloud and local LLMs sit behind one provider interface.
Auditable	Every automated action has an event record, reason, actor, and result.

1. Target Architecture

JAX should be built as a central core plus lightweight device nodes. The core owns memory, planning, permissions, agent orchestration, and APIs. Device nodes observe only approved context and expose narrowly scoped tools.

Recommended stack

Layer	V1 choice	Why
Core API	Python 3.12 + FastAPI	Strong agent/tool ecosystem; easy local services.
Database	Supabase Postgres + pgvector	Relational memory plus semantic retrieval.
Queue	Postgres job table initially	Avoid infrastructure complexity in V1.
Desktop node	Python + Windows APIs	Fastest route to active-window, filesystem and Git context.
Dashboard	Next.js + TypeScript	Good desktop/mobile control surface.
Phone	PWA first; native Android later	Ship command center quickly.
Local model	Ollama or LM Studio adapter	Private/offline tasks where useful.
Cloud models	Provider adapters	Use best model per task without lock-in.
Secrets	OS keychain / environment + server secret store	Never put credentials in JAX memory.

2. Repository Layout

```
jax/
  core/
    app/
      main.py
      config.py
      db.py
      models.py
      schemas.py
      auth.py
      policy.py
      memory.py
      retrieval.py
      planner.py
      orchestrator.py
      providers/
        base.py
        openai_provider.py
        local_provider.py
      tools/
        registry.py
        filesystem.py
        git_tools.py
        browser_context.py
      agents/
        coding.py
        research.py
        task_manager.py
        daily_brief.py
      routes/
        events.py
        memory.py
        chat.py
        tools.py
        devices.py
        handoffs.py
  node-windows/
    node.py
    collectors/
      active_window.py
      filesystem.py
      git_state.py
      browser.py
    local_policy.py
    spool.py
  web/
    app/
    components/
    lib/
  sql/
    001_schema.sql
    002_indexes.sql
    003_rls.sql
  tests/
  .env.example
  docker-compose.yml
  README.md
```

The node never receives blanket authority over the computer. Each collector and tool is separately enabled in configuration. Observation is not execution.

3. Permission Model

Level	Capability	Examples
0 - Observe	Read approved metadata/context	active app, repo branch, selected folders
1 - Organize	Write JAX-owned data	memory, summaries, tasks, handoffs
2 - Prepare	Create drafts/artifacts	code patch, email draft, report
3 - Execute	Run pre-approved reversible actions	open project, run tests, create file in sandbox

Level	Capability	Examples
4 - Confirm	Explicit approval every time	send message, deploy, delete, purchase, account changes
Blocked	Never autonomous	password extraction, secret exfiltration, disabling security controls

4. Database Schema

Do not use a single 'memory' table for everything. JAX needs structured state plus semantic retrieval.

Table	Purpose / important fields
users	id, display_name, timezone
devices	id, user_id, name, platform, last_seen, trust_level
workspaces	id, user_id, name, type, status
projects	id, workspace_id, name, path_hint, repo_url, status
events	id, device_id, project_id, type, app, summary, metadata_json, occurred_at
memories	id, user_id, kind, content, confidence, source_event_id, created_at, expires_at
memory_embeddings	memory_id, embedding vector
tasks	id, project_id, title, status, priority, due_at, source
decisions	id, project_id, decision, rationale, decided_at, supersedes_id
handoffs	id, from_device, project_id, summary, next_action, created_at
tool_permissions	user_id, device_id, tool_name, level, enabled
tool_runs	id, tool_name, arguments_hash, reason, approval_state, result, created_at
agent_runs	id, agent_name, objective, status, model, started_at, finished_at
documents	id, project_id, title, source, text_ref, checksum
daily_summaries	user_id, date, accomplishments, blockers, next_actions

Starter SQL

```
create extension if not exists vector;

create table devices (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null,
  name text not null,
  platform text not null,
  trust_level int not null default 0,
  last_seen timestamptz
);

create table events (
  id bigserial primary key,
  user_id uuid not null,
  device_id uuid references devices(id),
  project_id uuid,
  type text not null,
  app text,
  summary text not null,
  metadata jsonb not null default '{}',
  occurred_at timestamptz not null default now()
);

create table memories (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null,
  kind text not null,
  content text not null,
  confidence real not null default 1.0,
  source_event_id bigint references events(id),
  created_at timestamptz not null default now(),
  expires_at timestamptz
);

create index events_user_time_idx
  on events(user_id, occurred_at desc);
```

5. JAX Core API - Starter Code

core/app/main.py

```
from fastapi import FastAPI
from .routes import events, chat, devices, handoffs

app = FastAPI(title="JAX Core", version="0.1.0")

app.include_router(events.router, prefix="/v1/events")
app.include_router(chat.router, prefix="/v1/chat")
app.include_router(devices.router, prefix="/v1/devices")
app.include_router(handoffs.router, prefix="/v1/handoffs")

@app.get("/health")
async def health():
    return {"status": "ok", "service": "jax-core"}
```

core/app/config.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    jax_api_token: str
    llm_provider: str = "local"
    local_model_url: str = "http://localhost:11434"

    class Config:
        env_file = ".env"

settings = Settings()
```

Event ingestion route

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from datetime import datetime
from ..auth import require_node
from ..db import insert_event

router = APIRouter()

class EventIn(BaseModel):
    device_id: str
    type: str
    app: str | None = None
    project_id: str | None = None
    summary: str
    metadata: dict = {}
    occurred_at: datetime

@router.post("")
async def create_event(evt: EventIn, _=Depends(require_node)):
    event_id = await insert_event(evt.model_dump())
    return {"id": event_id, "accepted": True}
```

6. Windows JAX Node

The desktop node should summarize approved context locally before sending it to the core. V1 should capture active application metadata, project/repository state, and changes inside explicitly approved folders.

node-windows/node.py

```
import time, requests
from datetime import datetime, timezone
from collectors.active_window import current_window
from local_policy import allowed_event

CORE = "http://127.0.0.1:8788/v1/events"
TOKEN = "LOAD_FROM_WINDOWS_CREDENTIAL_MANAGER"
DEVICE_ID = "YOUR_REGISTERED_DEVICE_ID"

def emit(event):
    if not allowed_event(event):
        return
    requests.post(
        CORE,
        headers={"Authorization": f"Bearer {TOKEN}"},
        json=event,
        timeout=10,
    ).raise_for_status()

last = None
while True:
    ctx = current_window()
    key = (ctx.get("app"), ctx.get("title"))
    if key != last:
        emit({
            "device_id": DEVICE_ID,
            "type": "active_context",
            "app": ctx.get("app"),
            "summary": ctx.get("summary"),
            "metadata": ctx.get("metadata", {}),
            "occurred_at": datetime.now(timezone.utc).isoformat()
        })
        last = key
    time.sleep(15)
```

Local privacy filter

```
DENY_APPS = {"1Password", "Bitwarden", "KeePass"}
DENY_TITLE_WORDS = {"password", "private key", "seed phrase"}

def allowed_event(event: dict) -> bool:
    app = (event.get("app") or "").lower()
    text = (event.get("summary") or "").lower()

    if any(x.lower() in app for x in DENY_APPS):
        return False
    if any(x in text for x in DENY_TITLE_WORDS):
        return False
    return True
```

Do not build a keylogger. Do not collect password fields, clipboard history by default, authentication tokens, private keys, or raw continuous microphone/video. JAX gets useful context without creating a surveillance database.

7. Project Detection and Handoffs

JAX should infer projects from approved folder paths, Git repositories, window titles and manually assigned workspace rules. Confidence must be visible; uncertain classifications should remain unassigned.

Git context collector

```
import subprocess
from pathlib import Path

def git_context(path: str):
    p = Path(path)
    if not (p / ".git").exists():
        return None

    def run(*args):
        return subprocess.check_output(
            ["git", "-C", str(p), *args],
            text=True, timeout=5
        ).strip()

    return {
        "branch": run("branch", "--show-current"),
        "status": run("status", "--short")[:4000],
        "last_commit": run("log", "-1", "--pretty=%h %s"),
    }
```

Handoff object

```
{
  "project": "Harbison Standard",
  "from_device": "Work PC",
  "completed": [
    "Updated lead routing",
    "Ran API tests"
  ],
  "blockers": [
    "Production CRON_SECRET still unverified"
  ],
  "files_changed": [
    "app/api/leads/route.ts"
  ],
  "next_action": "Verify Vercel production env and rerun cron",
  "confidence": 0.94
}
```

The handoff is one of JAX's highest-value features: stop on one device, resume on another without reconstructing the previous session manually.

8. Memory Engine

Memory writes should be deliberate. Raw events are history; memories are distilled facts or useful state. JAX should not convert every observation into permanent memory.

Memory type	Retention	Example
Working	Hours/days	Currently debugging Vercel deployment.
Episodic	Long-term if useful	On Sep 20 Jason completed JAX schema V1.
Project	Project lifetime	Harbison Standard uses Vercel + Supabase.
Decision	Indefinite until superseded	Use local-first structured event collection.
Preference	Long-term	Prefer full-file code replacements.
Task	Until completed/cancelled	Test phone notification bridge.
Document knowledge	As source exists	Architecture PDF and project docs.

Retrieval algorithm

```
def retrieve_context(query, project_id=None):
    semantic = vector_search(query, project_id, limit=12)
    recent = recent_events(project_id, hours=24, limit=20)
    decisions = active_decisions(project_id, limit=10)
    tasks = open_tasks(project_id, limit=10)

    return rerank_and_trim(
        query=query,
        candidates=semantic + recent + decisions + tasks,
        token_budget=6000
    )
```

Memory-write policy

```
SAVE when:
- user explicitly asks JAX to remember something
- a durable project decision is made
- a task/blocker materially changes
- a repeated preference is confirmed
- a session handoff needs continuity

DO NOT SAVE when:
- content contains secrets/passwords
- observation is trivial or ephemeral
- confidence is low and not verified
- data belongs to an excluded application/folder
```

9. Agent Orchestration

JAX is the orchestrator. Specialist agents should be stateless workers that receive an objective, bounded context, tools, and permission ceiling.

Agent	Responsibility	Default max permission
Coding	Inspect code, propose patches, run tests	Level 3 in approved repos
Research	Public research and source synthesis	Level 1
Task Manager	Tasks, blockers, prioritization	Level 2
Daily Brief	Morning/night summaries	Level 1
Business	CRM/project analysis	Level 2
Trend	Trend intelligence engine	Level 1
Computer	Approved GUI/tool operations	Level 3; Level 4 for consequential actions
QA/Security	Review plans/tool calls before risky execution	No execution

Tool registry pattern

```
from dataclasses import dataclass
from typing import Callable

@dataclass
class Tool:
    name: str
    min_level: int
    handler: Callable

TOOLS = {}

def register(tool: Tool):
    TOOLS[tool.name] = tool

async def invoke(name, args, permission_level):
    tool = TOOLS[name]
    if permission_level < tool.min_level:
        raise PermissionError(name)
    return await tool.handler(**args)
```

Policy before execution

```
PLAN -> RETRIEVE CONTEXT -> SELECT TOOL
     -> POLICY CHECK
          -> safe + authorized: execute
          -> confirmation required: ask Jason
          -> blocked: refuse action
     -> RECORD RESULT -> UPDATE TASK/MEMORY
```

10. Model Provider Interface

Do not hard-code JAX to one model. Route tasks according to privacy, cost, latency and capability.

```
from abc import ABC, abstractmethod

class ModelProvider(ABC):
    @abstractmethod
    async def complete(self, messages, tools=None, **kwargs):
        ...

class LocalProvider(ModelProvider):
    async def complete(self, messages, tools=None, **kwargs):
        # Call Ollama/LM Studio-compatible local endpoint.
        ...

class CloudProvider(ModelProvider):
    async def complete(self, messages, tools=None, **kwargs):
        # Call configured cloud model through server-side credentials.
        ...
```

Routing policy

Task	Preferred route
Sensitive local summarization	Local model when capable
Complex coding/reasoning	Strong cloud model or best configured provider
Embedding generation	Local or low-cost embedding model
Routine classification	Small/fast model
High-impact action planning	Strong model + QA/security review

JAX identity prompt

```
You are JAX, Jason's private personal AI operating layer.

Priorities:
1. Preserve factual accuracy and provenance.
2. Maintain continuity across approved projects and devices.
3. Reduce Jason's cognitive switching cost.
4. Surface blockers, unfinished work and high-value next actions.
5. Never treat access as permission to act.
6. Never expose credentials or collect excluded private data.
7. Distinguish observation, inference and recommendation.
8. Use specialist agents/tools when they are better suited.
9. Record consequential actions in the audit log.
10. Ask for confirmation whenever policy requires it.
```

11. JAX Daily

Morning brief should be generated from actual project state, not generic productivity advice.

```
MORNING BRIEF

Active projects: 4

1. Harbison Standard
   Blocker: production cron authentication
   Next action: verify Vercel env
   Revenue relevance: high

2. Trend Intelligence Engine
   Last state: YouTube collector operational
   Next action: store metric snapshots

Overnight:
- 1 failed deployment
- 3 new tasks
- 0 security alerts

Recommended first block:
Harbison Standard - resolve deployment blocker.
```

Night brief

```
NIGHT HANDOFF

Completed:
- 4 tasks
- 2 code changes
- 1 research report

Unresolved:
- Android notification bridge
- Vercel cron secret

Tomorrow:
1. Resolve cron
2. Test JAX handoff on second PC
3. Connect phone PWA

No high-risk actions executed without confirmation.
```

Scheduling

Run daily briefs server-side. Device nodes should not have to remain open. Store each brief so JAX can compare planned work with actual work over time.

12. Phone Architecture

V1 should be a secure mobile web/PWA command center. Native Android can follow once the interaction model is proven.

Feature	V1
Chat/voice input	Send command to JAX Core
Share to JAX	Send URL/text/photo/file into selected project
Notifications	Blockers, failed jobs, requested reminders, approvals
Quick capture	Remember this / create task / attach to project
Device handoff	View and resume latest handoff
Approval center	Approve/deny Level-4 actions
Daily brief	Morning/night summary
Status	Which devices are online and current project state

Never make the phone a secret backdoor into the PCs. Remote actions go through the same permission policy and audit log as local actions.

13. Security & Privacy

Control	Implementation
Authentication	Per-device tokens; rotate/revoke independently
Transport	TLS outside localhost/VPN
Secrets	OS credential store/server secret manager; never memory DB
Data minimization	Structured summaries instead of continuous raw capture
Allow lists	Approved folders, repos, apps and tools
Deny lists	Password managers, banking, auth dialogs, private folders
Audit	Immutable-ish tool/action log with timestamps and reason
Kill switch	Disable a node or all execution tools immediately
Backups	Encrypted DB backup + tested restore
Deletion	Ability to purge project/event/memory classes deliberately

14. Environment & Setup

.env.example

```
DATABASE_URL=postgresql://...
JAX_API_TOKEN=replace_me
JAX_USER_ID=uuid
LLM_PROVIDER=local
LOCAL_MODEL_URL=http://localhost:11434
CLOUD_MODEL_API_KEY=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
JAX_ALLOWED_ORIGINS=http://localhost:3000
```

Python dependencies

```
fastapi
uvicorn[standard]
pydantic
pydantic-settings
asyncpg
sqlalchemy
pgvector
httpx
requests
python-jose[cryptography]
psutil
pywin32
watchdog
GitPython
```

Local development commands

```
# Core
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 8788 --reload

# Web dashboard
npm install
npm run dev

# Windows node
python node.py
```

Deployment rule

Keep JAX Core private whenever possible. For multi-device access, prefer a private network/VPN or strongly authenticated HTTPS endpoint. Do not expose a raw local-agent control port directly to the public internet.

15. Build Plan - Exact Order

Phase	Build	Exit criterion
0	Create repo, threat model, permission matrix	Know exactly what JAX may observe/execute
1	Supabase/Postgres schema + FastAPI core	Events can be authenticated and stored
2	First Windows node	Active context + Git state reaches core
3	Project classifier + dashboard	JAX correctly groups activity by project
4	Memory engine	Durable decisions/tasks can be retrieved
5	Chat + model provider	JAX answers using project/event context
6	Handoff engine	Stop on PC A, resume accurately on PC B
7	Second Windows node	Cross-device state proven
8	Tool registry + policy engine	Safe Level 0-3 tools work with audit trail
9	JAX Daily	Morning/night brief generated from real state
10	Phone PWA	Capture, chat, approvals, notifications
11	Specialist agents	Coding/research/task/trend agents routed by JAX
12	Hardening	Backups, revocation, kill switch, RLS, tests
13	Optional native Android	Deeper device integration only if needed

Definition of V1 complete

JAX is V1-complete when two computers can securely report approved context to one core; JAX can identify the active project, remember decisions/tasks, answer questions grounded in recent work, produce a reliable handoff, generate daily briefs, and require confirmation before consequential actions.

16. What to Build First This Week

Day 1: repository, Supabase project, schema, FastAPI health/event endpoints.

Day 2: Windows node, device registration, active-window collector, local privacy filters.

Day 3: Git/project context, event dashboard, project assignment rules.

Day 4: memory tables, retrieval, explicit 'remember this', tasks and decisions.

Day 5: JAX chat endpoint with contextual retrieval and provider abstraction.

Day 6: handoff generator and second-computer installation.

Day 7: security review, audit log, backups, kill switch and first JAX Daily report.

17. Expansion Roadmap

After the core works, connect the systems you are already building as JAX skills: Harbison Standard project intelligence, the Trend Intelligence Engine, COAI prospect analysis, coding/deployment workflows, document research, calendar/email connectors, and eventually controlled computer-use automation.

The architecture should remain simple at the top:

```
YOU
 |
JAX
 |
 +-- Memory
 +-- Projects
 +-- Devices
 +-- Agents
 +-- Tools
 +-- Automations
 +-- Daily intelligence
 |
 +--> everything else is a capability, not a separate assistant
```

This is the design target: one persistent personal AI identity with multiple specialized capabilities, shared memory, cross-device continuity, and explicit control boundaries.